

Aprendiendo PyTorch

Trabajo Fin de Máster
Máster en Ingeniería Informática



Universidad de Oviedo

Author:

Rubén Martínez Ginzo, UO282651@uniovi.es

Octubre 2025

Índice

1	Introducción	3
2	Sistema de desarrollo	4
2.1	Sistema operativo	4
2.2	Visual Studio Code	4
2.3	Jupyter Notebook	4
3	<i>Python</i>	5
3.1	Entornos virtuales	5
3.1.1	<i>pyenv</i>	5
3.2	<i>pip</i>	6
4	CUDA y drivers NVIDIA	7
4.1	Instalación de drivers NVIDIA	7
4.2	Instalación de CUDA	7
5	<i>PyTorch</i>	8
5.1	Instalación	8
5.1.1	Instalación de <i>ipykernel</i>	8
5.2	Tensores	9
5.3	Operaciones con tensores	9
5.4	Representación de imágenes con tensores	10
5.5	Tensores en GPU	10

Índice de figuras

Índice de tablas

Índice de Ecuaciones

1	Representación formal de un tensor de orden n	9
2	Propiedad de linealidad en operaciones con tensores.	9

Índice de Fragmentos

1	Instalación de una versión específica de <i>Python</i> con <i>pyenv</i>	5
2	Creación de un entorno virtual con <i>pyenv</i>	5
3	Activación de un entorno virtual con <i>pyenv</i>	5
4	Instalación de un paquete con <i>pip</i>	6
5	Verificación de la instalación de CUDA y los drivers de NVIDIA	7
6	Comando de instalación de <i>PyTorch</i> con soporte CUDA 13.0	8
7	Código de prueba de instalación de <i>PyTorch</i>	8
8	Salida esperada del código de prueba de instalación de <i>PyTorch</i>	8
9	Comando de instalación de <i>h5py</i>	8
10	Instalación de <i>ipykernel</i> en un entorno virtual de <i>pyenv</i>	8

1. Introducción

En el marco de este Trabajo Fin de Máster (TFM), surge la necesidad de desarrollar soluciones computacionales de alto rendimiento aplicadas al procesamiento digital de imágenes. Con el crecimiento exponencial de los datos y la complejidad de los algoritmos modernos, es de gran importancia aprovechar las capacidades de las unidades de procesamiento gráfico (GPU) para acelerar las tareas computacionales intensivas. Este documento tiene como objetivo introducir los fundamentos necesarios para comprender el uso de *PyTorch* una de las bibliotecas de computación científica más utilizadas en la actualidad.

El enfoque principal de este documento se centrará en los tensores de *PyTorch*, los cuales constituyen la estructura de datos fundamental para representar información en esta biblioteca. Entender su creación, manipulación y operaciones básicas es imprescindible para avanzar hacia algoritmos más complejos.

2. Sistema de desarrollo

Para trabajar de manera eficiente con *PyTorch* y aprovechar la aceleración mediante GPU, es necesario contar con un sistema de desarrollo adecuado. Este abarca tanto el sistema operativo como las herramientas necesarias para gestionar dependencias y bibliotecas externas.

2.1. Sistema operativo

Aunque *PyTorch* ofrece compatibilidad multiplataforma, el ecosistema de desarrollo más utilizado para aplicaciones de computación acelerada por GPU es Linux. Las distribuciones de Linux proporcionan un entorno muy flexible y personalizable, ampliamente compatible con las bibliotecas y controladores necesarios para trabajar con *PyTorch*. A lo largo de este documento se asumirá el uso de un sistema operativo basado en Linux —concretamente Fedora Linux 42¹—, aunque las instrucciones pueden adaptarse fácilmente a otros sistemas operativos como Windows o macOS.

2.2. Visual Studio Code

2.3. Jupyter Notebook

¹www.fedoraproject.org

3. Python

*Python*² es un lenguaje de programación interpretado, de alto nivel y propósito general. En los últimos años, se ha consolidado como uno de los lenguajes más populares en la comunidad científica y en el procesamiento de datos, debido a su simplicidad, sintaxis clara y gran variedad de bibliotecas especializadas. *PyTorch*, al ser una biblioteca de *Python*, requiere el uso de este lenguaje para su desarrollo y ejecución.

3.1. Entornos virtuales

El uso de entornos virtuales es fundamental para el desarrollo ordenado de proyectos en *Python*. Permiten aislar dependencias y versiones de paquetes entre distintos proyectos sin interferencias. Para crear entornos virtuales se puede utilizar la herramienta nativa *venv*³ o herramientas avanzadas como *virtualenv*⁴ o *conda*⁵. En este proyecto se opta por *pyenv*.

3.1.1. *pyenv*

*pyenv*⁶ es una sencilla herramienta de gestión de versiones de *Python*. Permite instalar múltiples versiones de *Python* y crear entornos virtuales asociados a cada una de ellas, facilitando el cambio entre versiones y la gestión de dependencias específicas para cada proyecto.

Con esta herramienta, puede instalarse cualquier versión de *Python*, tal como se puede observar en el Fragmento 1.

```
1 pyenv install <python_version>
```

Fragmento 1: Instalación de una versión específica de *Python* con *pyenv*

Se pueden crear entornos virtuales asociados a una versión específica de *Python*, como se muestra en el Fragmento 2.

```
1 pyenv virtualenv <python_version> <env_name>
```

Fragmento 2: Creación de un entorno virtual con *pyenv*

Y se puede activar un entorno virtual previamente creado con el comando del Fragmento 3.

```
1 pyenv activate <env_name>
```

Fragmento 3: Activación de un entorno virtual con *pyenv*

Este será el método empleado para gestionar las versiones de *Python* y los entornos virtuales a lo largo de este TFM.

²<https://www.python.org/>

³<https://docs.python.org/es/3/library/venv.html>

⁴<https://virtualenv.pypa.io/en/latest/>

⁵<https://docs.conda.io/projects/conda/en/stable/>

⁶<https://github.com/pyenv/pyenv>

3.2. *pip*

*pip*⁷ es el gestor de paquetes oficial de *Python*. Permite instalar, actualizar y eliminar paquetes y bibliotecas de *Python* desde el Python Package Index (PyPI) y otros índices de paquetes. Es una herramienta esencial para gestionar las dependencias de los proyectos en *Python*. Permite instalar paquetes de manera sencilla mediante comandos en la terminal, como se muestra en el Fragmento 4.

```
1 pip install <package_name>
```

Fragmento 4: Instalación de un paquete con *pip*

Este será el gestor de paquetes utilizado para instalar las bibliotecas necesarias en este TFM.

⁷<https://pip.pypa.io/en/stable/>

4. CUDA y drivers NVIDIA

CUDA⁸ (Compute Unified Device Architecture) es la plataforma de computación paralela de NVIDIA⁹ que permite ejecutar código y algoritmos en la GPU, aprovechando su capacidad de procesamiento masivamente paralelo. *PyTorch* utiliza CUDA para acelerar las operaciones de tensores y cálculos matemáticos, por lo que es fundamental contar con una instalación adecuada de CUDA y los drivers de NVIDIA en el sistema de desarrollo.

4.1. Instalación de drivers NVIDIA

Para instalar los drivers de NVIDIA en un sistema Linux, se pueden seguir los pasos detallados en la documentación oficial de NVIDIA¹⁰. Es importante asegurarse de que la versión del driver sea compatible con la versión de CUDA que se va a instalar posteriormente.

4.2. Instalación de CUDA

La instalación de CUDA puede realizarse descargando el instalador desde la página oficial de NVIDIA¹¹. Es recomendable seguir las instrucciones específicas para la distribución de Linux utilizada en el sistema de desarrollo.

Después de la instalación, es importante verificar que CUDA y los drivers de NVIDIA funcionan correctamente con el comando especificado en el Fragmento 5.

```
1 nvidia-smi
```

Fragmento 5: Verificación de la instalación de CUDA y los drivers de NVIDIA

⁸<https://developer.nvidia.com/cuda-zone>

⁹<https://www.nvidia.com/>

¹⁰<https://docs.nvidia.com/datacenter/tesla/tesla-installation-notes/index.html>

¹¹<https://developer.nvidia.com/cuda-downloads>

5. *PyTorch*

*PyTorch*¹² es una biblioteca de cómputo científico diseñada principalmente para el desarrollo de algoritmos de aprendizaje automático, aunque su estructura y diseño la convierten en una herramienta muy versátil para el cálculo numérico general y el procesamiento de datos de alta dimensión.

5.1. Instalación

Redactar...

```
1 pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu130
```

Fragmento 6: Comando de instalación de *PyTorch* con soporte CUDA 13.0

```
1 import torch
2 x = torch.rand(5, 3)
3 print(x)
4 print(torch.cuda.is_available())
```

Fragmento 7: Código de prueba de instalación de *PyTorch*

```
1 tensor([[0.6557, 0.0198, 0.0169],
2         [0.0468, 0.2888, 0.0026],
3         [0.4243, 0.1107, 0.6735],
4         [0.1006, 0.6152, 0.9174],
5         [0.1755, 0.9582, 0.8951]])
6 True
```

Fragmento 8: Salida esperada del código de prueba de instalación de *PyTorch*

```
1 pip install h5py
```

Fragmento 9: Comando de instalación de *h5py*

5.1.1. Instalación de *ipykernel*

```
1 pip install ipykernel
```

Fragmento 10: Instalación de *ipykernel* en un entorno virtual de *pyenv*

¹²<https://pytorch.org/>

5.2. Tensores

Los *tensores* son la base fundamental de *PyTorch*. Se trata de entidades multidimensionales capaces de representar información numérica de forma compacta y eficiente, permitiendo la ejecución de operaciones matemáticas optimizadas tanto en CPU como en GPU. Desde un punto de vista matemático, un tensor puede entenderse como una generalización de los vectores y las matrices a espacios de dimensión superior.

Como se muestra en la Ecuación 1, un tensor de orden n puede expresarse como:

$$T_{i_1 i_2 \dots i_n} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n} \quad (1)$$

Ecuación 1: Representación formal de un tensor de orden n

donde cada índice i_k recorre una dimensión d_k del espacio de datos. Así, un escalar puede considerarse un tensor de orden 0, un vector un tensor de orden 1 y una matriz un tensor de orden 2. Esta generalización permite extender las operaciones algebraicas clásicas a dominios de mayor complejidad, conservando propiedades como la linealidad y la composicionalidad.

Los tensores en *PyTorch* no solo almacenan datos, sino que también encapsulan información sobre su tipo de dato (`dtype`), dispositivo de ejecución (`device`) y forma (`shape`). Esta estructura interna los convierte en objetos altamente versátiles para el procesamiento numérico y gráfico, especialmente en contextos donde el rendimiento computacional es de vital importancia.

5.3. Operaciones con tensores

La biblioteca proporciona una amplia gama de transformaciones algebraicas y funcionales que pueden ejecutarse de manera vectorizada, evitando la iteración explícita y aprovechando la paralelización inherente al hardware.

Las operaciones básicas incluyen la suma, resta, multiplicación escalar, producto punto y producto matricial. Además, *PyTorch* permite realizar operaciones más avanzadas como transposición, cambio de forma (`reshape`), concatenación y reducción, todas optimizadas internamente para ejecutarse con la máxima eficiencia posible.

Desde el punto de vista algebraico, estas operaciones preservan las propiedades fundamentales de la linealidad, de modo que para dos tensores A y B y un escalar α se cumple que, como se muestra en la Ecuación 2:

$$\alpha(A + B) = \alpha A + \alpha B \quad (2)$$

Ecuación 2: Propiedad de linealidad en operaciones con tensores

El modelo de ejecución de *PyTorch* está diseñado para que todas las operaciones sobre tensores sean consistentes en términos de dimensiones y tipos, lo que facilita el diseño modular de algoritmos numéricos. Además, el motor interno de la biblioteca realiza comprobaciones automáticas de compatibilidad entre dimensiones, lanzando excepciones cuando las operaciones no son válidas.

5.4. Representación de imágenes con tensores

[...]

5.5. Tensores en GPU

[...]